

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Commands

```
uv sync                                # install deps (+ dev group)
uv run pytest                          # full suite (132 tests, ~1s)
uv run pytest tests/unit/test_schema.py # one file
uv run pytest -k "auto_limit"          # by test name pattern
uv run pytest tests/integration/test_mcp_server.py::test_query_database_end_to_end # one t

uv run ruff check src/ tests/ eval/    # lint (auto-fix with --fix)
uv run mypy src/                       # type check (strict)

uv run python tests/fixtures/build_sample_db.py # regenerate gitignored sample.db
uv run python -m eval.runner --limit 5        # smoke-run eval (needs API key)

uv run nl-db query "... " --db <path>      # CLI entry
uv run nl-db-mcp --db <path>                # MCP stdio server (NL-on-MCP)
uv run nl-db-mcp --db <path> --expose-run-sql [--allow-writes] # MCP with raw-SQL tool
uv run nl-db-ui                             # Streamlit playground (NL-on-MCP)
```

Architecture

nl-db is a Python NL-to-SQL pipeline exposed through three surfaces — a CLI, an MCP stdio server, and a Streamlit playground — all built on the same core. Three layered abstractions matter; the rest is mechanical:

1. **LLMProvider Protocol (src/nl_db/llm/provider.py)**. Pipeline code is forbidden from importing `anthropic` or `openai` directly — every LLM call goes through this Protocol. `llm/registry.py::build_provider(settings)` is the only place vendor SDKs are touched. Adding a new backend (Apple Intelligence shim, Ollama, etc.) is one file in `llm/` plus one branch in the registry. New backends speaking the OpenAI wire format go through `openai_compatible.py` without code changes — they're a config-only addition.
2. **SchemaExtractor Protocol (src/nl_db/schema/base.py)**. SQLite is the only implementation in v1. `schema/cache.py` caches results keyed by (path, mtime) so repeated CLI/MCP calls don't re-introspect the DB. `render_for_prompt()` turns a `Schema` into the compact form injected into

LLM prompts — token efficiency is intentional, don't expand it without measuring.

3. **Pipeline** (`src/nl_db/pipeline.py`). Orchestrates: `schema()` → `build_sql_prompt` → `generate_outcome` → branch on outcome. The `generate_outcome` call returns a three-state `GenerationOutcome` (`Answer(sql)` | `CannotAnswer(reason, available_tables)` | `Clarify(question)`) defined in `src/nl_db/generator.py`. Only the `Answer` branch runs `validate_sql` → optional `paraphrase_sql` → `confirm` callback → `execute`. `CannotAnswer` and `Clarify` short-circuit; the Pipeline injects `available_tables` from the live schema on `CannotAnswer`. `PipelineOutput.state` returns the stable string "ANSWER" | "CANNOT_ANSWER" | "CLARIFY" for callers that don't want to `isinstance`-check. The `ConfirmFn` callback (`Answer` branch only) is the seam between UX surfaces: the CLI plugs an interactive `rich.Confirm`, the MCP server returns SQL to the host model (auto-confirms), `--no-confirm` skips. Tuning knobs (`temperature`, `max_output_tokens`, `paraphrase_temperature`, `paraphrase_max_output_tokens`, `auto_limit`, `num_few_shot`) are constructor kwargs sourced from the `GenerationConfig` block in `Settings`.

Invariants (do not break)

- **Schema-first prompting.** Every SQL generation includes the live (or cached) schema. No stale snapshots, no schema-less prompts.
- **Three-state generator output.** Every NL question resolves to exactly one of `Answer(sql)`, `CannotAnswer(reason, available_tables)`, or `Clarify(question)`. The system prompt (`prompts/system.py`) defines the wire format (fenced SQL, or `CANNOT_ANSWER: ... / CLARIFY: ...` sentinels). `parse_outcome()` in `generator.py` does the dispatch.
- **NL-friendly errors.** Raw exceptions (`sqlglot.ParseError`, `sqlite3.OperationalError`, `SQLValidationError`, vendor SDK errors) are translated by `nl_db.nl_errors.humanize()` before reaching any user-facing surface. The pipeline still raises raw exceptions internally — only CLI / UI / MCP wrappers humanize.
- **SQL transparency.** Generated SQL is surfaced (to the user via CLI, to the host LLM via MCP tool response) before any side effect. The paraphrase step (`prompts/paraphrase.py`) gives a one-sentence NL explanation as a second mitigation — schema is deliberately NOT re-sent in the paraphrase prompt.
- **Read-only by default.** `validator.validate_sql` uses `sqlglot`'s parse tree (not regex) to detect `INSERT/UPDATE/DELETE/DROP/ALTER/TRUNCATE` and refuses unless `allow_writes=True`. Auto-LIMIT is injected for unbounded `SELECTs` (skipped for destructive statements).

- **Provider-agnostic by construction.** No file outside `llm/` may know which LLM is in use. The provider name is *configuration*, not code.
- **Eval-driven.** `eval/dataset.yaml` is the NL→SQL test set against `tests/fixtures/sample.db`. 35 cases total: 30 ANSWER cases (row/SQL-pattern scored) + 3 CANNOT_ANSWER + 2 CLARIFY cases (state-scored). Cases declare `expected_state` (defaults to ANSWER). Any prompt change in `src/nl_db/prompts/` or system-prompt change should be re-evaluated via `python -m eval.runner` before merging.

MCP server

`src/nl_db/mcp/server.py` uses FastMCP (stdio transport). Default surface: four tools (`list_tables`, `describe_database`, `describe_schema`, `query_database`) and two Resources (`db://schema` for the full schema, `db://schema/<table>` per-table). A fifth tool `run_sql` is registered only when the server is started with `--expose-run-sql` (the NL-first design treats SQL execution as a power-user escape hatch, not the default surface). `--allow-writes` requires `--expose-run-sql` and exits with code 2 otherwise. `query_database` returns one of three response shapes keyed by a "state" field: ANSWER (with `sql`, `paraphrase`, `columns`, `rows`, ...), CANNOT_ANSWER (with `reason`, `available_tables`), or CLARIFY (with `question`). It also accepts an optional `conversation_id` string — calls sharing an id participate in a multi-turn conversation; the server holds a per-process `dict[str, Conversation]` keyed on that id (no cross-process persistence). **Tool descriptions in `src/nl_db/mcp/tools.py` are product copy** — host LLMs read them to decide when and how to call each tool. Treat edits there like prompt engineering, not boilerplate. `run_sql`'s `readOnlyHint/destructiveHint` annotations flip based on the `--allow-writes` flag when the tool is registered.

Conversation state

`src/nl_db/conversation.py` defines `Turn` (frozen dataclass: `question`, `outcome`, optional one-line `row_summary`) and `Conversation` (mutable list of turns). `Conversation.to_prompt_context(max_turns=5)` renders recent turns as a compact text block; SQL is included verbatim, but result rows are NEVER dumped — only the summary line from `summarize_rows()` (`columns` + `row count` + truncated first row). `build_sql_prompt()` accepts optional `history: Conversation | None`, placed between few-shot examples and the current question. `Pipeline.run()` accepts the same parameter. The schema always comes first in the prompt; history is bounded so it can't crowd the schema out. Two consumers: the Streamlit Chat tab (per `st.session_state.chat_conversation`) and the MCP server's `conversation_id`-keyed dict.

Config precedence

Env vars > `.env` > `./nl-db.toml` (project-local, `NL_DB_CONFIG_FILE` override) > built-in defaults. Implemented via a custom `PydanticBaseSettingsSource` in `config.py` (the obvious approach of passing TOML data as kwargs to `Settings()` makes TOML beat env — which is wrong). `Settings` has four blocks: `provider`, `db`, `limits`, `generation`. API keys come from `ANTHROPIC_API_KEY` / `OPENAI_API_KEY` / `OPENAI_COMPATIBLE_API_KEY` — never from TOML. `save_settings()` writes the four non-secret blocks (provider sans `api_key`, `db`, `limits`, `generation`) but explicitly never the API key. `nl-db schema` and `nl-db config` deliberately don't build a provider so they work without an API key.

Apple Intelligence path

Not built in this repo. `nl-db` consumes Apple Intelligence through *any* third-party Swift HTTP shim that exposes Apple's `FoundationModels` framework via the OpenAI `/v1/chat/completions` wire format. Point `NL_DB_PROVIDER_BASE_URL` at the shim — `nl-db` treats it as just another `openai_compatible` provider. No special code path.

Streamlit playground

`src/nl_db/web/app.py` is a UI on top of the same `Pipeline`. Five tabs: **Query**, **Chat** (scaffolded only — real multi-turn lands in roadmap item 7), **Schema**, **History**, **About**. Every sidebar control maps to a `Pipeline` kwarg — `temperature`, `max_output_tokens` (SQL and paraphrase separately), `auto_limit`, `num_few_shot`, `max_rows`, `timeout_s`, `allow_writes`, `paraphrase` on/off. Sidebar edits are session-scoped (`st.session_state`) until the user clicks **Save to disk**, which writes `./nl-db.toml` via `save_settings()`; the API key is never persisted. The `Query` tab's primary action is **Ask** — one button that runs the full pipeline (`pipeline.run(..., confirm=lambda: True)`) and shows the result dataframe first; the SQL + paraphrase live in a collapsible “How I answered” expander below the result. This matches the project framing (`nl-db` queries the database in NL, not “converts NL to SQL”). For destructive SQL, the validator still blocks unless **Allow writes** is checked in the sidebar — that toggle is the safety gate, not a separate confirmation click. `CannotAnswer` renders an info banner with available tables; `Clarify` renders a yellow warning with a follow-up text input that re-runs with the clarification appended. Sidebar sub-widgets that gate on a checkbox (paraphrase temperature, paraphrase max tokens) are always rendered with `disabled=True/False` rather than conditionally — Streamlit clears widget-key `session_state` when a widget isn't rendered, which breaks downstream readers. The `Query` tab also has a **Preview only** button that

builds the would-be HTTP request body without making the call, and a **Debug** expander that shows both the actual wire request **and** the LLM's response — built on `_make_capturing_http_client()`, an `httpx.Client` with `request` + `response` event hooks injected into the openai/anthropic SDK via the existing `client=` constructor parameter. The response hook calls `response.read()` to cache the body and then `_extract_response_text()` pulls the LLM's text content out of the provider-specific shape (OpenAI `choices[0].message.content`, Anthropic `content[*].text`); this is what surfaces when SQL extraction fails so you can verify what the model actually returned. `web/` is excluded from mypy (Streamlit's mutable `session_state` pattern fights strict typing).

Testing patterns

- **Never hit a real LLM in tests.** Provider tests use fake clients with the vendor SDK shape (`tests/unit/test_llm.py`). Pipeline/CLI/MCP tests use a `CannedProvider` that returns pre-programmed strings.
- `tests/conftest.py` strips provider env vars and `NL_DB_*` at the start of every test for isolation, and builds the sample DB on demand (session-scoped).
- **MCP tool tests** invoke `server._tool_manager.get_tool(name).run(arguments=...)` directly — no subprocess. Resource tests use `server.read_resource(uri)`.

Commit conventions

Conventional commits (`feat(scope): ...`, `chore: ...`, `docs: ...`, `fix: ...`). Existing global git config is the authoritative author identity — do not set per-repo `user.name/user.email`, do not add Co-Authored-By trailers.

Roadmap and future work

`plan.md` is the authoritative roadmap. The next-iteration sequence is:

1. Three-state generator output (`ANSWER` | `CANNOT_ANSWER(reason, available_tables)` | `CLARIFY(question)`) + NL-error wrapping
2. MCP `describe_database()` tool + top-level `db://schema` Resource; tool descriptions nudge schema-first grounding
3. `CANNOT_ANSWER` carries hints (available tables, suggested rephrase)
4. MCP `run_sql` moves behind `--expose-run-sql` (default off); `query_database` becomes the canonical NL path
5. Eval coverage for the new states
6. UI three-state rendering + a Chat tab in the Streamlit playground
7. Multi-turn chat with optional `conversation_id` on MCP `query_database`

Future work beyond the roadmap (caching strategy, Postgres/MySQL adapters, schema enrichment, result summarizer, cost/observability, transport hardening) is detailed in `plan.md`. Postgres/MySQL adapters drop into the existing `SchemaExtractor` Protocol (`schema/base.py`) and the `executor.QueryExecutor` Protocol — slots are already wired.